

DIGITAL NOTES

ON

WEB TECHNOLOGIES

III B.TECH – I SEM



Prepared by

● **DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING**
Sreyas Institute of Engineering and Technology
An Autonomous Institution

Approved by AICTE, Affiliated to JNTUH, Accredited by NAAC-A Grade, NBA (CSE, ECE & ME) & ISO 9001:2015 Certified
Bandlaguda, Nagole, Hyderabad, Telangana 500068

Unit 2

HTML Common tags- List, Tables, images, forms, Frames; Cascading Style sheets;

XML: Introduction to XML, Defining XML tags, their attributes and values, Document Type Definition, XML Schemes, Document Object Model, XHTML Parsing XML Data – DOM and SAX Parsers in java.

INTRODUCTION

HTML stands for **HyperText Markup Language**. It is used to structure the content on the web by using various elements (commonly known as tags). These HTML elements define the different sections of a web page, such as headings, paragraphs, links to other webpages, listings, images, tables, etc. These elements tell the browser about the content and formatting to display.

HTML is HyperText + Markup Language. Where,

- **HyperText** refers to the way in which Web pages (HTML documents) are linked together. Thus, the link available on a webpage is called "HyperText".
- **Markup Language**, which means you use HTML to simply "mark up" a text document with tags that tell a Web browser how to structure it to display.

HTML Element

An HTML element is a basic building block to create a webpage, and It is created by a start tag, content, and end tag. In an HTML element, the content is placed between a start and end tag.

The basic syntax of an HTML element is –

`<tag_name>content</tag_name>`

Consider the following example demonstrates an HTML element –

`<h1>It is top-level heading</h1>`

Here,

`<h1>` is the start tag.

"It is top-level heading" is the content, which is placed inside the start and end tags.

`</h1>` is the closing tag.

STRUCTURE OF THE HTML PROGRAM:-

The HTML Program is generally divided into two sections i.e head and body. We use `<head>` and `<body>` tags to indicate these two sections. `<head>` section holds the header information of a webpage document indicated by a title that is provided by using `<title>` tag in the `<head>`. The title helps us to refer to the webpage. `<body>` section contains the content which we want to display within the webpage. Anything that is not a tag will be displayed within the webpage.

Example:

```
<html>
<head>
<title>First Webpage</title>
</head>
<body>
This is my first page
</body>
</html>
```

BASIC TAGS IN HTML

HTML provides following basic tags to work with hypertext documents.

- **<html> tag:**

This is the basic tag of HTML document. By using this tag the [browser](#) can identifies the weather it is html document or not.

format :

```
<html>
</html>
```

- **<head> tag:**

It indicates the first part of HTML document, and it contains control information and title of document.

format :

```
<head>
----
----
</head>
```

- **<title> tag:**

It is used to display title of document.

format :

```
<title>
title of document
</title>
```

- **<body> tag:**

It indicates the second part of HTML document, and it contains content that displayed on screen and tags which control how that content is formatted by browser.

Body tag has different parameters which includes background,bgcolor...

background: specifies an image as background for web page.

bgcolor: specifies color as background for web page.

format :

```
<body bgcolor="#f5f5f5">
```

content of body

```
</body>
```

- **
 tag :**

It is used to break the current line (or) moves to next line.

format :

```
<br/> or <br>
```

- **<hr/> tag :**

It is used to insert horizontal rule or line.

format :

```
< hr [ width=n ] [ size=n ] [ align="left" | "center" | "right" ] />
```

Example 1: "welcome.html"

```
<html>
```

```
<head>
```

```
<title>Welcome</title>
```

```
</head>
```

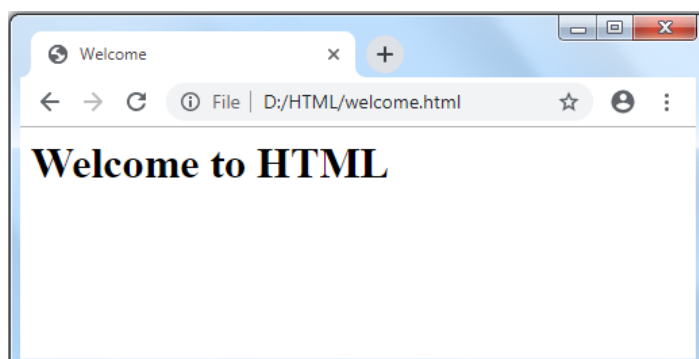
```
<body>
```

```
<h1>Welcome to HTML</h1>
```

```
</body>
```

```
</html>
```

Output:



Blocks in HTML

- **<p> tag:**

It is used to display paragraph in HTML document. And it has common parameter i.e "align".

format :

```
<p [ align="left" | "center" | "right" ] >
```

```
</p>
```

- **Heading tags :**

These are simple forms of text formatting that vary text sizes based on header level.

Those are

format :

```
<h1> ----</h1>      Max size
```

```
<h2>-----</h2>      |
```

```
<h3>-----</h3>      |
```

```
<h4>-----</h4>      |
```

```
<h5>-----</h5>      v
```

```
<h6>-----</h6>      Min size
```

Example 2: "blocks.html"

```
<html>
```

```
<head>
```

```
<title>Blocks Example</title>
```

```
</head>
```

```
<body bgcolor="#9ae5e5">
```

```
<h1 align="center">HTML</h1>
```

```
<hr/>
```

```
<p> HTML stands for Hyper Text Markup Language,It used to create static web pages.<br/>
```

```
HTML supports different types of tags those are html,head,title,h1,h2 and etc..</p>
```

```
Heading tags are <br/>
```

```
<h1>HTML</h1>
```

```
<h2>HTML</h2>
```

```
<h3>HTML</h3>
```

```
<h4>HTML</h4>
```

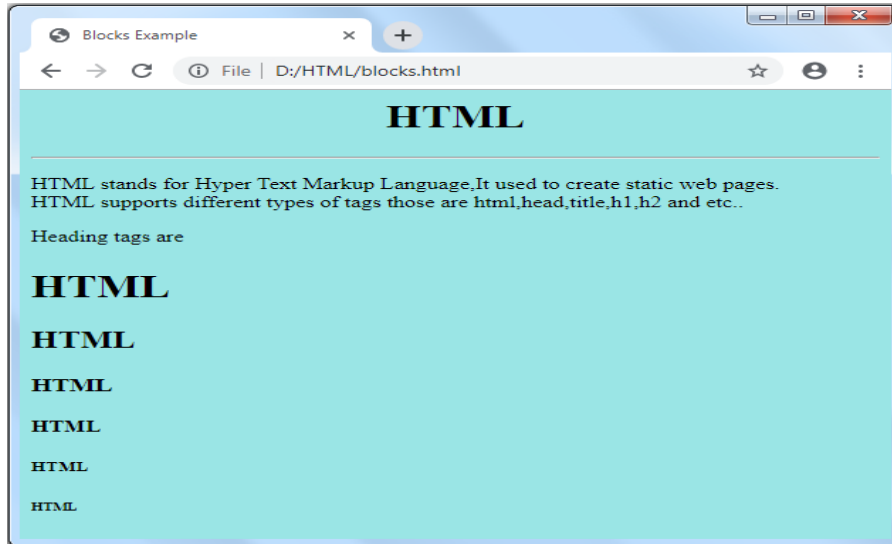
```
<h5>HTML</h5>
```

```
<h6>HTML</h6>
```

```
</body>
```

```
</html>
```

Output:



Text Formatting Tags in HTML

HTML provides following tags to work with text in html documents.

- ** tag:**

By using this tag, we can be able to change the size and color of the text.

format :

```
<font [size=n] [color="#rrggbb"] > text </font>
```

- ** tag:**

It bolds the text. i.e it is used to display the written text in bold format.

format :

```
<b> some text </b>
```

- ** tag :**

It bolds the text and it is an advanced version of tag.

format :

```
<strong> some text </strong>
```

- **<u> tag:** It underlines the text.

format :

<u> some text </u>

- **<ins> tag:**

It underlines the text and it is an advanced version of <u> tag

format :

<ins> some text </ins>

- **<i> tag:**

It italics the text

format :

<i> some text </i>

- ** tag:**

It italics the text and it is an advanced version of <i> tag.

format :

 some text

- **<s> tag :**

It strikes the text.

format :

<s> some text </s>

- ** tag :**

It strikes the text and it is an advanced version of <s> tag.

format :

 some text

- **<mark> tag :**

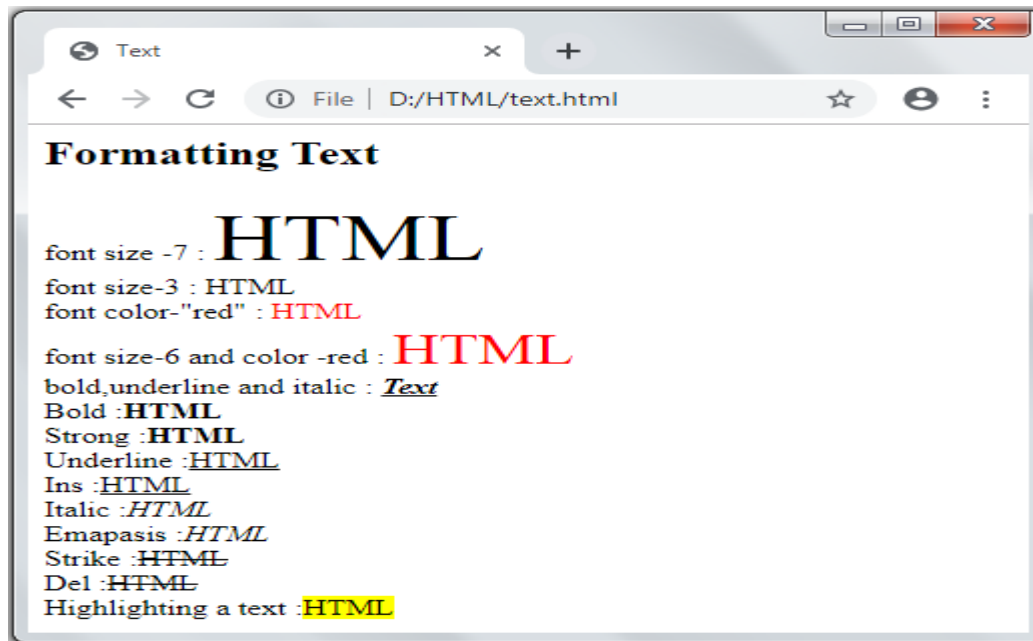
It used to highlight a text in HTML.

format :

<mark> some text </mark>

Example 1: "text.html"

```
<html>
<head>
<title>Text</title>
</head>
<body>
<h2>Formatting Text</h2>
font size -7 : <font size="7">HTML</font><br/>
font size-3 : <font size="3">HTML</font><br/>
font color-"red" : <font color="red">HTML</font><br/>
font size-6 and color -red : <font size="6" color="red">HTML</font><br/>
bold,underline and italic : <b><u><i>Text</i></u></b><br/>
Bold :<b>HTML</b><br/>
Strong :<strong>HTML</strong><br/>
Underline :<u>HTML</u><br/>
Ins :<ins>HTML</ins><br/>
Italic :<i>HTML</i><br/>
Emapasis :<em>HTML</em><br/>
Strike :<s>HTML</s><br/>
Del :<del>HTML</del><br/>
Highlighting a text :<mark>HTML</mark>
</body>
</html>
```


Output:

- **<sub> tag :**

It sub scripts the text. Ex: H₂O

format :

H₂O

- **<sup> tag :**

It super scripts the text. Ex: a²

format :

a²

Example 2: "subsup.html"

<html>

<body>

Formula for Water : H₂O

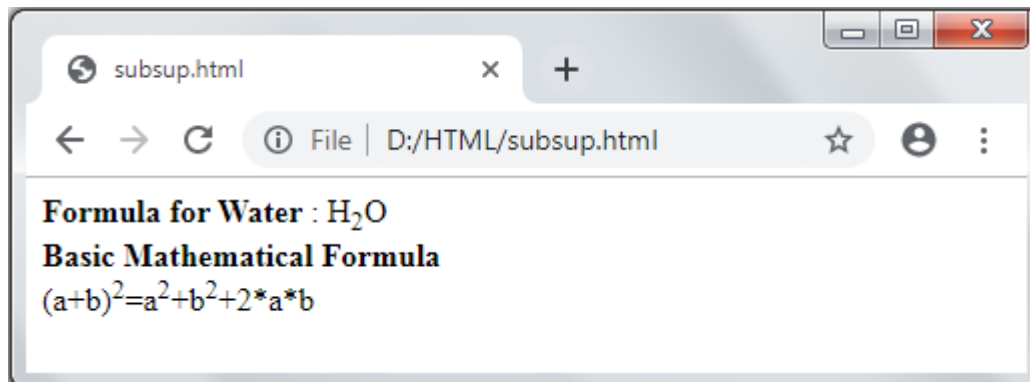
Basic Mathematical Formula

(a+b)²=a²+b²+2*a*b

```
</body>
```

```
</html>
```

Output:



Character Escape Sequences :

Escapes sequences could be wrongfully interpreted as markup in HTML, so the following characters are reserved in HTML and must be replaced with their corresponding HTML entities:

- " is replaced with "& quot;"
- & is replaced with "& amp;"
- < is replaced with "& lt;"
- > is replaced with "& gt;"
- space is replaced with "& nbsp;"

Anchor Tag in HTML

- The most important capability of HTML is its ability to **create hyperlinks**. Hyperlinks can be applied to either text or images.
- Anchor tag is used to create link between two documents, Minimum it requires a parameter is "**href**" which indicates destination document.

format:

```
<a href="filename/path" [alt="text"] > -----</a>
```

"**href**" is mandatory argument and it specifies the path or url of the document.

"**alt**" is optional argument and it specifies the alternative text.

- Sometimes images can be used as hyperlinks.

format:

```
<a href="filename/path" [alt="text"] ></a>
```

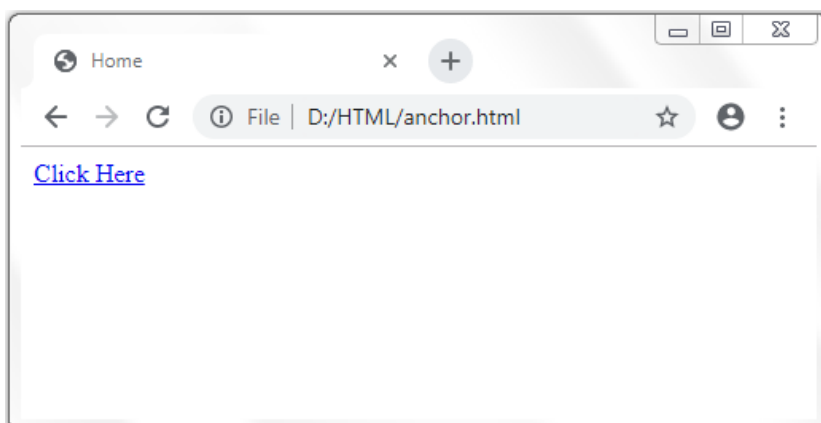
Example : *"anchor.html"*

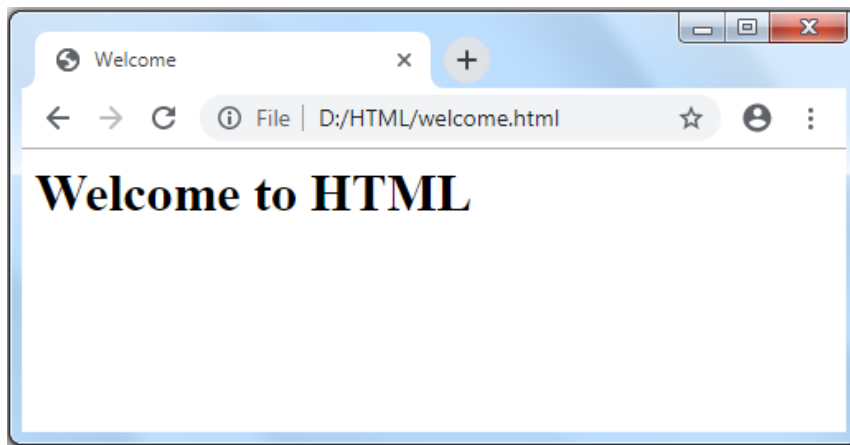
```
<html>
<head>
<title>Home</title>
</head>
<body>
<a href="welcome.html"> Click Here</a>
</body>
</html>
```

Example : *"welcome.html"*

```
<html>
<head>
<title>Welcome</title>
</head>
<body>
<h1>Welcome to HTML</h1>
</body>
</html>
```

Output:





Marquee Tag in HTML

The Marquee tag is a non-standard HTML element which is used to scroll a image or text horizontally or vertically. In simple words, you can say that it scrolls the image or text up, down, left or right automatically.

Marquee's tag contains several attributes,some of them are

- **behavior:**It facilitates user to set the behavior of the marquee to one of the three different types: scroll, slide and alternate.
- **direction:** defines direction for scrolling content. It may be left, right, up and down.
- **width:** defines width of marquee in pixels or %.
- **height:** defines height of marquee in pixels or %.
- **bgcolor:** defines background color.

format:

```
<marquee [behavior="scroll"|"slide"|"alternate"] [direction="left"|"right"|"up"|"down"] [width=n%]
[height=n%] [bgcolor="#rrggbb"]> some text or image </marquee>
```

Example : "marquee.html"

```
<html>
<head>
<title>Marquee</title>
</head>
<body>
```

Basic Marquee:


```
<marquee>Welcome to html marquee </marquee>
```

Scroll Marquee:


```
<marquee width="100%" behavior="scroll" bgcolor="pink">
```

This is an example of a scroll marquee...

```
</marquee>
```

Slide Marquee:


```
<marquee width="100%" behavior="slide" bgcolor="pink">
```

This is an example of a slide marquee...

```
</marquee>
```

Alternate Marquee:


```
<marquee width="100%" behavior="alternate" bgcolor="pink">
```

This is an example of a alternate marquee...

```
</marquee>
```

Direction in marquee :


```
<marquee width="100%" direction="right">
```

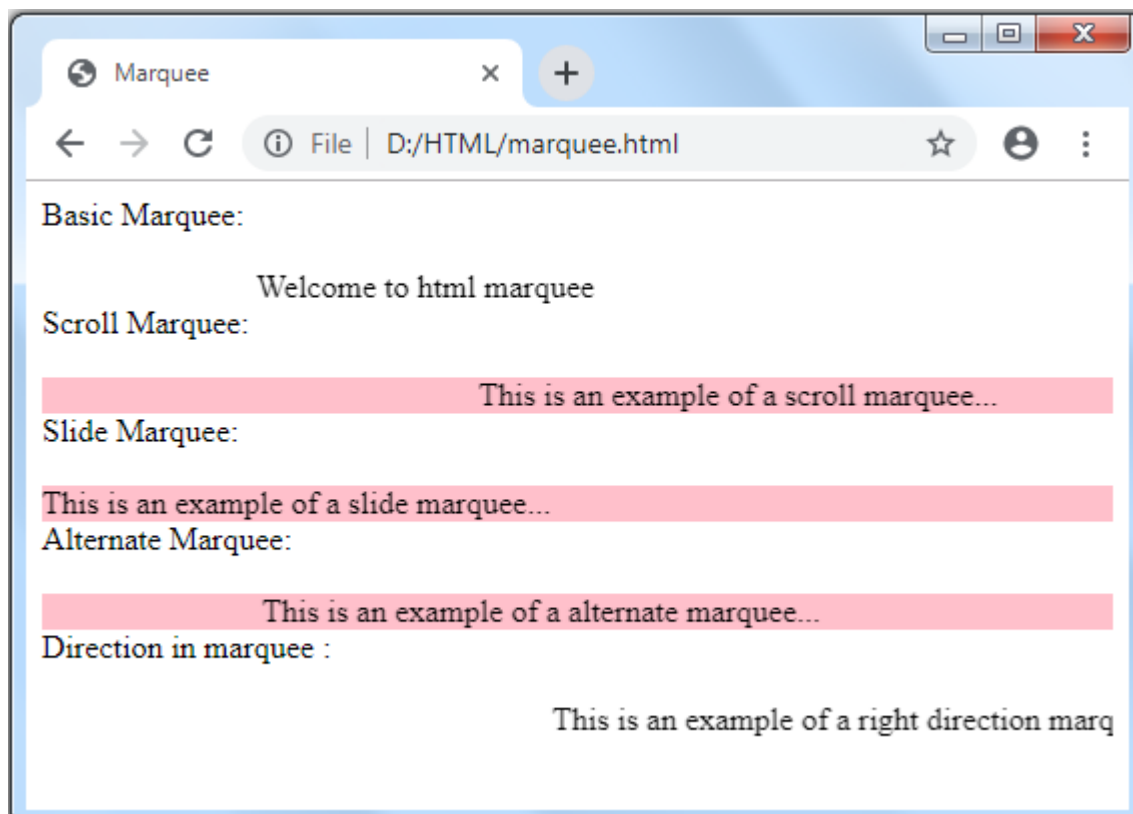
This is an example of a right direction marquee...

```
</marquee>
```

```
</body>
```

```
</html>
```

Output:



LISTS

HTML lists allow web developers to group a set of related items in lists.

Unordered HTML List

An unordered list starts with the `<ul>` tag. Each list item starts with the `<li>` tag. The list items will be marked with bullets (small black circles) by default:

Example

```
<ul>
  <li>Coffee</li>
  <li>Tea</li>
  <li>Milk</li>
</ul>
```

The HTML `<ul>` tag defines an unordered (bulleted) list.

Unordered HTML List - Choose List Item Marker

The CSS `list-style-type` property is used to define the style of the list item marker. It can have one of the following values:

Value	Description
disc	Sets the list item marker to a bullet (default)
circle	Sets the list item marker to a circle
square	Sets the list item marker to a square
none	The list items will not be marked

Disc

Example - Disc

```
<ul style="list-style-type:disc;">
  <li>Coffee</li>
  <li>Tea</li>
  <li>Milk</li>
</ul>
```

Circle

```
<ul style="list-style-type:circle;">
```

Square

```
<ul style="list-style-type:square;">
```

No List Marker

```
<ul style="list-style-type:none;">
```

Nested HTML Lists

Example

```
<ul>
  <li>Coffee</li>
  <li>Tea
    <ul>
      <li>Black tea</li>
      <li>Green tea</li>
    </ul>
  </li>
  <li>Milk</li>
</ul>
```

Ordered HTML List

An ordered list starts with the tag. Each list item starts with the tag.

The list items will be marked with numbers by default:

Ordered HTML List - The Type Attribute

Type	Description
type="1"	The list items will be numbered with numbers (default)
type="A"	The list items will be numbered with uppercase letters
type="a"	The list items will be numbered with lowercase letters
type="I"	The list items will be numbered with uppercase roman numbers
type="i"	The list items will be numbered with lowercase roman numbers

Numbers:

```
<ol type="1">
  <li>Coffee</li>
  <li>Tea</li>
  <li>Milk</li>
</ol>
```

Uppercase Letters

```
<ol type="A">
```

Lowercase Letters

```
<ol type="a">
```

Roman Numbers - Uppercase

```
<ol type="I">
```

Roman Numbers - Lowercase

```
<ol type="i">
```

Control List Counting

By default, an ordered list will start counting from 1. If you want to start counting from a specified number, you can use the start attribute:

Example

```
<ol start="50">
```

Nested HTML Lists

Lists can be nested (list inside list):

Example

```
<ol>
  <li>Coffee</li>
  <li>Tea
    <ol>
      <li>Black tea</li>
      <li>Green tea</li>
    </ol>
  </li>
  <li>Milk</li>
</ol>
```

3. Definition List (<dl> ----- </dl>) :

- The definition list is used to specify list of terms and their definitions.
- It has the following inbuilt tags

<dl> tag specifies the definition list.

<dt> tag specifies the defined term.

<dd> tag specifies the definition for term.

Example :

```
<dl>
<dt>HTML</dt>
<dd>HTML is Markup language</dd>
```



```
<dt>XML</dt>
```

```
<dd>XML is an extended markup language</dd>
```

```
</dl>
```

Output:

HTML

HTML is Markup language

XML

XML is an extended markup language

TABLE TAGS IN HTML

- A table is a matrix of rows and columns. The HTML table provides a highly readable way of presenting many kinds of information.i.e by using table we can present information in structuring format.
- HTML tables are used to manage the layout of the page e.g. header section, navigation bar, body content, footer section etc. but it is recommended to use div tag over table to manage the layout of the page.

The basic table tags:

- **<table>-----</table>**

It indicates the starting of the table.The most common attribute for <table> tag is border,And it has different attributes like height,width.....,

- **<th>-----</th>**

It is used to give table heading(column header).

- **<tr>-----</tr>**

It is used to start a table row.

- **<td>-----</td>**

It indicates a table data or a cell.

- **<caption>-----</caption>**

It is used to insert caption into table.

format :

```
<table [ align="center"|"left"|"right" ] [ border=n px ] [ height=n% ] [ width=n% ] [ cellpadding=n px ] [ cellspacing= n px ] >
```

```
<caption> ----- </caption>
```

```
<tr>
```

```
<th> ---- </th>
```

```

</tr>

<tr>

<td> ---- </td>

</tr>

</table>

```

Commonly used arguments in table

- **align:** It specifies alignment of table.i.e, left or center or right.
- **border:** It specifies border of table in pixels.
- **height:** It sets amount of screen that table will use in the factor of height.
- **width:** It sets amount of screen that table will use in the factor of width.
- **cellpadding:** It specifies the space between content of cell and its border.
- **cellspacing:** It specifies the space between content of cells.
- **colspan:** It is used combine two or more columns.
- **rowspan:** It is used to combine two or more rows.

Example : "table.html"

```

<html>

<head>

<title>Table</title>

</head>

<body>

<table align="center" border=1 width="30%" cellpadding=10px>

<caption> Employee Details</caption>

<tr>

<th>EID</th>

<th>ENAME</th>

<th>AGE</th>

</tr>

<tr>

<td>101</td>

<td>Madhu</td>

<td>32</td>

</tr>

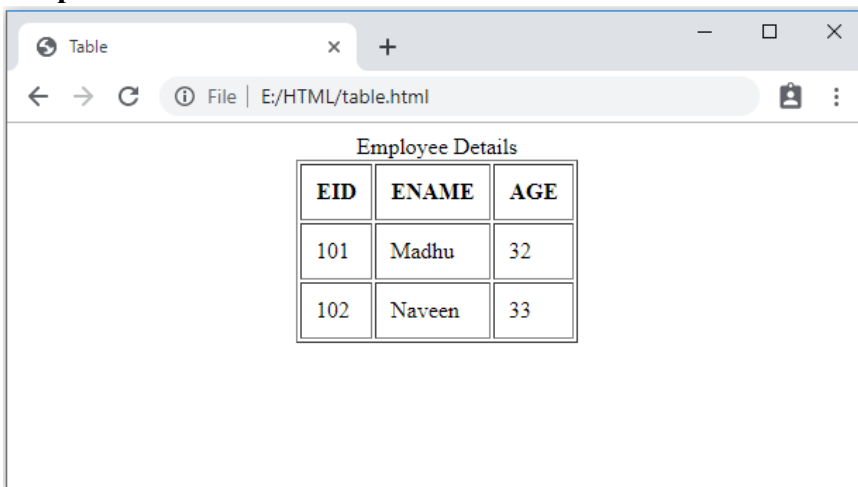
```

```

<tr>
<td>102</td>
<td>Naveen</td>
<td>33</td>
</tr>
</table>
</body>
</html>

```

Output:



EID	ENAME	AGE
101	Madhu	32
102	Naveen	33

HTML Table with colspan :

If you want to combine more than one column, you can use the colspan attribute.
Let's see the example that span or combine two columns

Example : "*tablecols.html*"

```

<html>
<head>
<title>Table-Colspan</title>
</head>
<body>
<table width="50%" border=1>
<tr>
<th>Name</th>
<th colspan="2">Mobile No.</th>

```

```

</tr>

<tr>

<td>B.Tech-CSE</td>

<td>1234567890</td>

<td>9876543210</td>

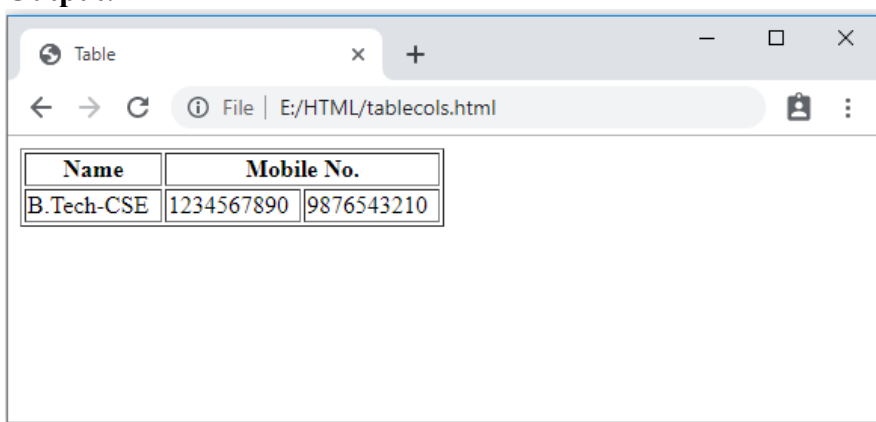
</tr>

</table>

</body>

</html>

```

Output:**HTML Table with rowspan :**

If you want to combine more than one row, you can use the rowspan attribute.
Let's see the example that span or combine two rows.

Example : *"tablerows.html"*

```

<html>

<head>

<title>Table-Rowspan</title>

</head>

<body>

<table width="50%" border=1>

<tr>

<th>Name</th>

<td>B.Tech-CSE</td>

```

```

</tr>

<tr>

<th rowspan="2">Mobile No.</th>

<td>1234567890</td>

</tr>

<tr>

<td>9876543210</td>

</tr>

</table>

</body>

</html>

```

Output:

Name	B.Tech-CSE
Mobile No.	1234567890
	9876543210

IMAGE TAG IN HTML

- Images play an important role in webpages. The total look of a webpage is improved using images.
- Images can be embedded inside html document using **** tag. Minimum it requires a parameter i.e **"src"** ,which indicates source of image.
- **** tag is used to display image on the web page. **** tag is an empty tag that contains attributes only, closing tags are not used in HTML image element.

format :

```

```

- Sometimes we can use images as hyperlink.

Example:

```
<a href="a.html" ></a>
```

Attributes of tag :

The **src** and **alt** are important attributes of HTML img tag. All attributes of HTML image tag are given below.

- **src:** It is a necessary attribute that describes the source or path of the image. It instructs the [browser](#) where to look for the image on the server. The location of image may be on the same directory or another server.
- **alt:** The alt attribute defines an alternate text for the image, if it can't be displayed. The value of the alt attribute describes the image in words.
- **width:** It is an optional attribute which is used to specify the width to display the image. It is not recommended now. You should apply CSS in place of width attribute.
- **height:** It specifies the height of the image. The HTML height attribute also supports iframe, image and object elements. It is not recommended now. You should apply CSS in place of height attribute.

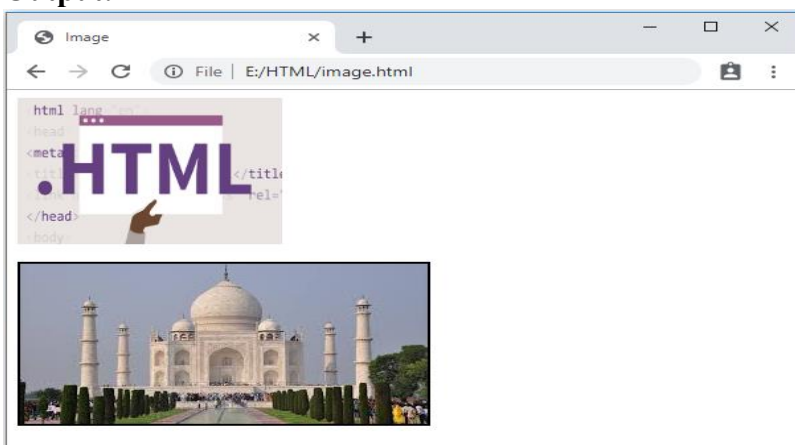
Example : "image.html"

```
<html>
<head>
<title>Image</title>
</head>
<body>

<br/><br/>

</body>
</html>
```

Output:



HTML FORMS

An HTML form is used to collect user input. The user input is most often sent to a server for processing.

Syntax:

<form>.....</form>

.

Form Elements

.

1. Text Box:

A text box allows the user to enter a single line of text.

Tag: <input type="text">

2. Password Box:

A password box hides the characters typed by the user for secure input.

Tag: <input type="password">

3. Checkbox:

A checkbox lets the user select one or more options from a set of choices.

Tag: <input type="checkbox">

4. Radio Button:

A radio button allows the user to select only one option from a group.

Tag: <input type="radio">

5. Dropdown List:

A dropdown list provides a menu from which the user can choose one option.

Tag: <select><option>...</option></select>

6. Text Area:

A text area lets the user enter multi-line text.

Tag: <textarea></textarea>

7. Submit Button:

A submit button sends the form data to the server for processing.

Tag: <input type="submit"> or <button type="submit">

8. Reset Button:

A reset button clears all input fields and returns them to default values.

Tag: <input type="reset">

9. File Upload:

A file upload field allows the user to upload a file from their device.

Tag: <input type="file">

10. GET Method:

The GET method appends form data to the URL as query parameters. It is suitable for retrieving or viewing data, like search or filter forms. The data is visible in the browser's address bar (not secure). GET requests can be bookmarked and shared easily. There is a limit to the amount of data that can be sent.

Example:

```
<html>
```

```
<form method="get" action="data.html">
```

```
<input type="text" name="username">
```

```
<input type="submit">
```

```
</form>
```

The navigation in the bar looks like : data.html?username=YourInput

11. POST Method:

The POST method sends form data in the body of the HTTP request. It is ideal for sending sensitive or large data, like login info or files. Data is not visible in the URL, offering better privacy. POST requests cannot be bookmarked or cached. There is no size limit on the data (more secure and flexible).

Example:

```
<html>
```

```
<form method="post" action="data.html" enctype="multipart/form-data">
```

```
<input type="text" name="username">
```

```
<input type="submit">
```

```
</form>
```

The navigation in the bar looks like : http://yoursite.com/data.html

Form Example:

```
<html>
```

```
<head>
```

```
<title>Sample Form</title>
```

```
</head>
```

```
<body bgcolor="#f9f9f9">
```

```
<h1><center><u>Student Registration Form</u></center></h1>
```

```
<form name="fi" method="get" action="data.html">
```

Name: <input type="text" name="username">

Password: <input type="password" name="password">

Hobbies:


```
<input type="checkbox" name="hobby" value="reading">
```

Reading


```
<input type="checkbox" name="hobby" value="sports"> Sports<br><br>
```

Gender:


```
<input type="radio" name="gender" value="male"> Male<br>
```

```
<input type="radio" name="gender" value="female"> Female<br><br> Branch:
```

```
<select name="branch">
```

```
<option value="cse">CSE</option>
```

```
<option value="ece">ECE</option>
```

```
<option value="eee">EEE</option>
```

```
</select><br><br>
```



```

Address:<br>
<textarea name="address" rows="4" cols="40"></textarea><br><br>
Upload Resume: <input type="file" name="resume"><br><br>
<input type="submit" value="Submit">
<input type="reset" value="Reset">
</form>
</body>
</html>

```

Data.html(output of the form stored in this file)

```

<html>
<head>
  <title>Submitted Data</title>
</head>
<body bgcolor="#f9f9f9">
  <h2><center><u>Submitted Student Details</u></center></h2>
  <table border="1" align="center" cellpadding="10">
    <tr>
      <th>Field</th>
      <th>Value</th>
    </tr>
    <tr>
      <td>Name</td>
      <td id="username"></td>
    </tr>
    <tr>
      <td>Password</td>
      <td id="password"></td>
    </tr>
    <tr>
      <td>Hobby</td>
      <td id="hobby"></td>
    </tr>
    <tr>
      <td>Gender</td>
      <td id="gender"></td>
    </tr>
  </table>

```

```

<tr>
  <td>Branch</td>
  <td id="branch"></td>
</tr>
<tr>
  <td>Address</td>
  <td id="address"></td>
</tr>
<tr>
  <td>Resume File</td>
  <td id="resume"></td>
</tr>
</table>

<script>
function getParam(name) {
  const url = new URL(window.location.href);
  return url.searchParams.get(name) || "";
}

document.getElementById("username").innerText = getParam("username");
document.getElementById("password").innerText = getParam("password");
document.getElementById("hobby").innerText = getParam("hobby");
document.getElementById("gender").innerText = getParam("gender");
document.getElementById("branch").innerText = getParam("branch");
document.getElementById("address").innerText = getParam("address");
const resumePath = getParam("resume");
const fileName = resumePath.split("\\").pop().split("/").pop();
document.getElementById("resume").innerText = fileName;
</script>
</body>
</html>

```

Output:

The screenshot shows a web browser window with multiple tabs. The active tab displays a form titled "Student Registration Form". The form fields include:

- Name:
- Password:
- Hobbies: ☐ Reading, ☐ Sports
- Gender: ☐ Male, ☐ Female
- Branch:
- Address:
- Upload Resume:
-

The browser's address bar shows the file path: C:/Users/dell/Desktop/HTML%20Examples/Formnew.html. The Windows taskbar at the bottom shows the time as 00:46 on 08-09-2023.

HTML FRAMES

- HTML frames allow a web developer to divide the browser window into multiple sections, where each section (called a frame) can independently load a different HTML document.
- Frames are defined using the `<frameset>` tag, which replaces the `<body>` tag when used.
- Each frame within a frameset is defined using the `<frame>` tag.
- Frames can be arranged horizontally using rows or vertically using cols.
- The layout can be specified using percentages (30%) or fixed pixel sizes (200px).
- For example, `rows="50%,50%"` divides the screen into two equal horizontal frames, while `cols="30%,70%"` creates a sidebar and content area.
- Frames also support nested framesets, allowing for more complex page layouts.
- The name attribute can be used to target specific frames from links using the target attribute.
- For example, a navigation frame on the left can open pages in the right content frame.

Examples

Example 1: Using cols to divide into two vertical sections:

```
<frameset cols="30%,70%">
  <frame src="left.html">
  <frame src="right.html">
</frameset>
```

Example 2: Using rows to divide into two horizontal sections:

```
<frameset rows="50%,50%">
  <frame src="top.html">
  <frame src="bottom.html">
```

```
</frameset>
```

Example of Frames (Gmail format webpage):

(1) Main.html

```
<html>
```

```
<frameset rows="80%,20%">
```

```
<frameset cols="20%,80%">
```

```
<frame src="menu.html" name="menu">
```

```
<frame src="gmail.html" name="content">
```

```
</frameset>
```

```
<frame src="drive.html" name="footer">
```

```
</frameset>
```

```
</html>
```

(2) menu.html

```
<!DOCTYPE html>
```

```
<html>
```

```
<body bgcolor="#d9edf7">
```

```
<h3>Menu</h3>
```

```
<ul>
```

```
<li><a href="inbox.html" target="content">  Inbox</a></li>
```

```
<li><a href="sent.html" target="content">  Sent</a></li>
```

```
<li><a href="drafts.html" target="content">  Drafts</a></li>
```

```
</ul>
```

```
</body>
```

```
</html>
```

(3) gmail.html

```
<html>
```

```
<body bgcolor="#fefbd8">
```

```
<h2>Welcome to Gmail</h2>
```

```
<p>This is a simulation layout for Gmail using HTML frames.</p>
```

```
</body>
```

```
</html>
```

(4) drive.html

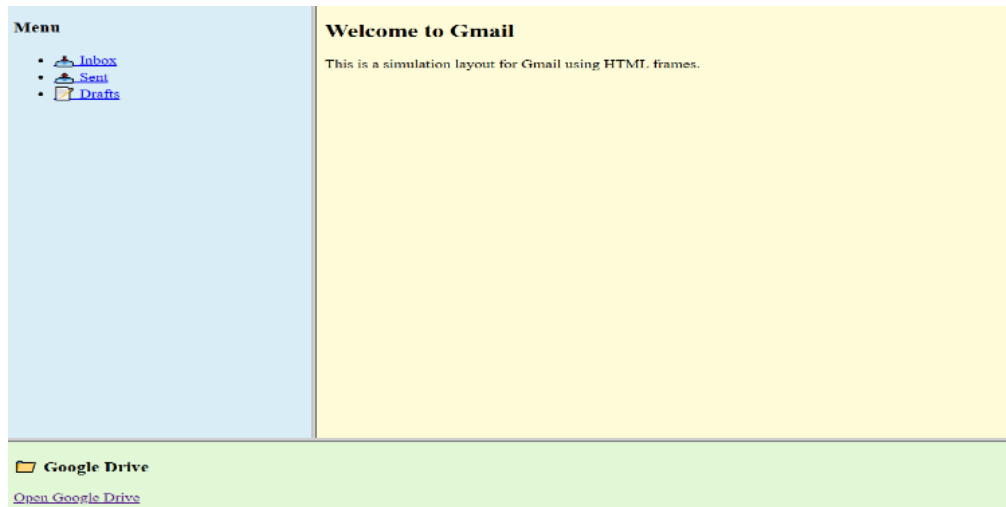
```
<html>
```

```
<body bgcolor="#e1f7d5">
```

```
<h3>📁 Google Drive</h3>
```

```
<a href="https://drive.google.com" target="_blank">Open Google Drive</a>
```

```
</body></html>
```



CASCADING STYLE SHEETS

- CSS stands for Cascading Style Sheets
- CSS (Cascading Style Sheets) is a language designed to simplify the process of making web pages presentable.
- It allows you to apply styles to HTML documents by prescribing colors, fonts, spacing, and positioning.
- HTML uses tags, and CSS uses rule sets.
- CSS styles are applied to the HTML element using selectors.

CSS Syntax:

HTML : `<h1>Web Technologies</h1>`

CSS : `h1 { color: blue; font-size: 12px; }`

Syntax: `selector {property: value; property: value;}`

Example: `h1 {color: blue; font-size: 12px;}`

CSS consists of style rules that are interpreted by the browser and applied to the corresponding elements.

A style rule set includes a **selector** and a **declaration** block.

- **Selector:** Targets specific HTML elements to apply styles.
- **Declaration:** Combination of a property and its corresponding value.

CSS types:

- 1. Inline CSS:** Directly within the HTML element using the style attribute.
- 2. Internal CSS:** Within a <style> tag in the <head> section.
- 3. External CSS:** The external CSS is the CSS linked to an HTML file using the <link> tag.

(1) Inline CSS

Inline CSS applies styles directly to HTML elements using the style attribute, allowing for quick, unique styling without external stylesheets.

Syntax:

```
<tag style = " "></tag>
```

Example:

```
<html>
<body>
<div style="text-align: center; font-family: Arial, sans-serif;">
  <h2>Welcome to My Website</h2>
  <p>Enjoy your stay!</p>
</div>
<p style="color: blue; font-size: 18px;">
  This is a paragraph styled with inline CSS.
</p>
<button style="background-color: #4CAF50; color: white; padding:
  10px 20px; border: none; border-radius: 5px;">
  Click Me
</button>
</body>
</html>
```

When to Use Inline CSS?

- **Applying Unique Styles:** Ideal for styling individual elements without affecting global styles.
- **Testing or Debugging:** Useful for quick style adjustments during development.

Advantages

- Using style attributes we can provide styles directly to our HTML elements.
- Inline CSS Overrides external and internal styles with higher specificity.
- No need to create and upload a separate document as in the external style.
- Inline styles have high specificity, allowing precise control over individual elements.
- Enables dynamic style changes using JavaScript or server-side logic.
- Inline styles don't require separate CSS files, potentially reducing HTTP requests.

Disadvantages

- Adding style attributes to every HTML element is time-consuming.
- Styling multiple elements can increase your page's size and download time, impacting overall page performance.
- Reduced separation of concerns between HTML structure and CSS.
- Inline styles cannot be used to style pseudo-elements and pseudo-classes.
- It can be difficult to maintain consistency and make global style updates.

(2)Internal CSS

Internal CSS is a method for defining CSS styles directly within an HTML document. It's particularly useful for applying unique styles to a single web page, and it's embedded within the <style> element located in the <head> section of the HTML file.

How to Use Internal CSS ?

To use internal CSS, you need to include CSS rules within a <style> tag inside the HTML document's <head>. This allows you to define styles by selecting HTML elements or classes and applying styling rules within the tag. The styles defined by internal CSS apply only to the specific web page where they are included.

Syntax:

```
<style>
  /* Internal CSS starts here */
</style>
```

Example: Here is the basic example of using internal CSS.

```
<html>
<head>
  <title>Internal CSS</title>
  <style>
    /* Internal CSS starts here */
    h1 {
      color: green;
      text-align: center;
      font-size: 50px;
    }
    p {
      font-size: 25px;
      color: blue;
      text-align: center;
```

```

    }
    .container {
        text-align: center;
    }
    .btn {
        background-color: red;
        color: white;
        border-radius: 5px;
        padding: 10px 20px;
        text-decoration: none;
    }
    .btn:hover {
        background-color: #0056b3;
    }
</style>
</head>
<body>
    <h1>WEB TECHNOLOGIES</h1>
    <p>A Computer Science Portal..!</p>
    <div class="container">
        <a href="https://www.sreyas.ac.in/"
            class="btn">Click Me</a>
    </div>
</body>
</html>

```

Advantages of Internal CSS

- **Localized Styling:** Keeps styles within the HTML file, avoiding conflicts with other pages and making it easier to manage styles at the local level.
- **Higher Specificity:** Internal CSS has higher specificity than external CSS, allowing easier overriding of external styles within the same HTML file.
- **Performance:** Reduces HTTP requests, potentially enhancing performance as no additional CSS files need to be loaded.
- **Ease of Implementation:** Simple to use and implement, making it easy to quickly apply styles to a single page.

Disadvantages of Internal CSS

- **Repetition:** Styles must be repeated in multiple HTML files if the same styles are needed across different pages.
- **Increased File Size:** Embedding CSS in HTML increases the file size of the HTML document.
- **Reduced Reusability:** Limited code reusability across different web pages.
- **Limited Management:** Managing styles across multiple pages can become difficult without a centralized stylesheet.

By understanding the strengths and limitations of internal CSS, you can make informed decisions about when and how to use it effectively in your web projects. For further learning, consider exploring more detailed CSS tutorials and examples.

(3)External CSS

External CSS is a method used to style multiple HTML pages with a single stylesheet. This approach involves creating a separate CSS file with a .css extension that contains style properties applied to various selectors (such as classes, IDs, headings, etc.). By using external CSS, you can maintain a consistent design across multiple web pages efficiently.

How to Link an External CSS to HTML

To link an external CSS file to an HTML document, you need to use the <link> element within the <head> section of your HTML file. The <link> element should have the rel attribute set to "stylesheet" and the href attribute specifying the path to your CSS file.

Syntax:

To link an external CSS file to an HTML document, you need to use the <link> element within the <head> section of your HTML file. The <link> element should have the rel attribute set to "stylesheet" and the href attribute specifying the path to your CSS file.

```
<link rel="stylesheet" href="path/to/your/styles.css">
```

Example : Here's another example demonstrating how to use external

Main.html

```
<html>
<head>
<link rel="stylesheet" href="mystyle.css">
</head>
<body>
<h1>This is a heading</h1>
<p>This is a paragraph.</p>
</body>
</html>
```

mystyle.css

```
body {
    background-color: lightblue;
}
```

```
h1 {
    color: navy;
    margin-left: 20px;
}
```

Advantages of External CSS

- **Improved Maintainability:** Having styles in a separate file makes it easier to manage and update your styles without modifying each HTML document.
- **Enhanced Reusability:** The same CSS file can be linked to multiple HTML files, promoting consistent design across your site.
- **Efficient Caching:** Browsers cache external CSS files, leading to faster page load times on subsequent visits.

Disadvantages of External CSS

- **Loading Time:** Pages may not render correctly until the external CSS file is fully loaded, potentially causing a flash of unstyled content (FOUC).
- **Performance Concerns:** Linking multiple CSS files can increase download times, affecting overall site performance.
- **Versioning and Caching Challenges:** Large-scale projects may encounter difficulties in versioning and caching, leading to inconsistencies in styles.

Linking external CSS to your HTML is a consistent and maintainable design across multiple web pages.

By following the steps outlined above, you can easily apply external styles to your HTML documents and enhance the visual appeal of your website. For a seamless experience, ensure that your CSS files are correctly linked and optimized.

CSS selectors**1. Element Selector**

The **element selector** is used to apply style to a specific HTML element. For example, if you use the `p` selector, every `<p>` tag in your HTML will be affected by the styles you define. This selector does not use any prefix like `.` or `#`. It's a simple and effective way to apply consistent styles to headings, paragraphs, or any other HTML tags. This is ideal for general formatting such as font size, color, alignment, or spacing across all instances of a tag.

Example:

```
p {  
  color: blue;  
  font-size: 16px;  
  text-align: center;  
  font-weight: bold;  
}
```

2. ID Selector

The **ID selector** is used to style a specific element that has a unique id attribute. You denote this selector with a hash symbol (#) followed by the ID name. Since IDs must be unique within a page, the styles defined here will apply to only that one element. This is helpful when you want to highlight or customize one unique section, like a navigation bar or a specific heading.

Example:

```
#header {  
  background-color: black;  
  color: white;  
  padding: 10px;  
  text-align: center;  
}
```

3. Group Selector

The **group selector** allows you to apply the same styles to multiple elements at once. This is done by separating the selectors with commas. It helps keep your code clean and avoids repeating the same styles for each element. Group selectors are particularly useful when several elements share the same styling, like heading levels or input fields.

Example:

```
h1, h2, h3 {  
  color: darkgreen;  
  font-family: Arial;  
  text-transform: uppercase;  
  text-align: left;  
}
```

4. Universal Selector

The **universal selector**, represented by an asterisk (*), targets **all** elements on the web page. It is commonly used for resetting default browser styles or setting basic styles like margin, padding, or box-

sizing for every element. While powerful, it should be used carefully as it applies styles globally and can impact performance on larger web pages.

Example:

```
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
  font-family: sans-serif;
}
```

5. Class Selector

The **class selector** is used to style elements that share a common class attribute. It is denoted with a dot (.) followed by the class name. Unlike ID selectors, classes can be reused on multiple elements, making this selector ideal for applying consistent styles across various parts of a page. It's useful for styling sections like boxes, buttons, or cards.

Example:

```
.box {
  border: 2px solid gray;
  padding: 20px;
  background-color: lightyellow;
  width: 200px;
}
```

EXTENSIBLE MARKUP LANGUAGES (XML)

Extensible Markup Languages (XML) history begins with the development of Standardised Generalised Markup Language (SGML) by Charles Goldfarb, along with Ed Mosher and Ray Lorie in the 1970s while working at IBM (Anderson, 2004). The purpose of SGML was to create vocabularies which could be used to mark up documents with structural tags.

XML is designed to store and transport data, it was created to provide an easy to use and store self describing data. XML became a W3C Recommendation on February 10, 1998. XML is not a replacement for HTML, it is designed to be self-descriptive. XML is designed to carry data, not to display data. XML tags are not predefined. You must define your own tags. XML is platform independent and language independent.

What is XML

- XML stands for Extensible Markup Language
- XML is a markup language much like HTML.
- XML was designed to carry data, not to display data.
- XML tags are not predefined. You must define your own tags.
- XML is designed to be self-descriptive.
- XML is W3C recommendation.

Representing Web data

XML stands for extensible Markup Language, developed by W3c in 1996. XML 1.0 was officially adopted as a W3c recommendation in 1998. XML was designed to carry data, not to display data. XML is designed to be self-descriptive. XML is an ISO-compliant subset of Standard Generalized Markup Language (SGML) than can be defined in your own tags. A meta language and tags describe the content. XML supports Cascading Style Sheets(CSS), eXtensible Stylesheet Language(XSL), Document Object Model (DOM).

Advantages

- XML is a simple scripting language where as humans can easily read.
- XML document is language natural that means one language programming code can generate and XML and document and these documents can be passed by other languages.

Goals of XML

- The user must to able to define and use his own tags.
- Allows the user to build his own tag library, based on his web requirement.
- Allow user to define the formatting rules for the user defined tags.
- XML must support storage or transport of data.

XML and HTML were designed with different goals

- XML is designed to carry data emphasizing on what type of data it is whereas HTML is designed to display data emphasizing on how data looks.
- XML tags are not predefined like HTML tags.
- XML provides a framework for defining markup languages whereas HTML is a markup language.
- XML is about carrying information, which makes it dynamic whereas HTML is about displaying data, hence it is static.

XML Syntax

XML tags are as simple as in HTML but in XML tags are more flexible. There are no predefined XML tags, You must define your personalized tags (user defined tags).

The simple XML document must contain opening tag and closing tag. The XML tags are case sensitive i.e. and both tags are different. The XML tags are used to define the scope of elements in XML document. The first element of XML document is called root element.

Start Tag

The beginning of every XML element is marked by a start-tag.

Syntax

```
<studyglance>
```

End Tag

Every element that has a start tag should end with an end-tag. The end tags include a forward slash ("/") before the name of an element.

Syntax

```
</studyglance>
```

Empty Tag

The text that appears between start-tag and end-tag is called content. An element which has no content is termed as empty. A start-tag immediately followed by an end-tag or a complete empty-element tag is as shown below

```
<!-- empty-element -->
```

```
<studyglance> </studyglance>
```

```
<!-- A complete empty-element -->
```

```
<studyglance />
```

Comments in XML

Comments are a piece of text placed within a program to help other users to understand it, which the computer ignores when running the program. XML Comments are used as notes or lines for understanding the purpose of an XML code

Syntax

```
<!-- This is a comment -->
```

Comments Rules

- Comments cannot appear before XML declaration.
- Comments may appear anywhere in a document.
- Comments must not appear within attribute values.
- Comments cannot be nested inside the other comments.

XML ELEMENTS

Elements are the basic building blocks of XML and all documents will have at least one element called root element that is the parent of all other elements. Once the XML prolog is finished you need to create the root element of the document. Everything else in the document lies under this element to form a hierarchical tree.

Syntax

```
<element-name attribute1 attribute2>
....content
</element-name>
```

The elements in an XML document form a document tree. The tree starts at the root and branches to the lowest level of the tree. All elements can have sub elements (child elements).

```
<root>
  <child>
    <subchild>.....</subchild>
  </child>
</root>
```

Underneath the root element can lie other elements. These nested elements can be used to show individual or repetitive items of data depending on what you are trying to represent. For example, your root element could be element underneath and the elements underneath could show the students's characteristics, such as **<name>** and **<rollno>**.

```
<department>
  <student>
    <name>.....</name>
    <rollno>....</rollno>
  </student>
</department>
```

XML Naming Rules

- Element names are case-sensitive
- Element names must start with a letter or underscore
- Element names cannot start with the letters xml (XML or Xml, etc)

- Element names can contain letters, digits, hyphens, underscores, and periods
- Element names cannot contain spaces

XML Attributes

Attributes are one of the two main building blocks of XML. Attributes are name-value pairs associated with an element. Attributes are part of XML elements. An element can have multiple unique attributes. Attribute gives more information about XML elements, they define properties of elements.

Syntax

```
<element-name attribute1="value1" attribute2="value2">
....content..
</element-name>
```

example

```
<student gender="female">
....content..
</student>
```

Attributes rules

- Attributes consist of a name and a value separated by an equals sign. The name follows the same rules as element names.
- The attribute value must be in quotes. You can use either single or double quotes, but you can't mix them in a single attribute
- There must be a value part, even you can't leave value just empty.
- Attribute names must be unique per element.

example

```
<department>
  <student gender="female">
    <name>.....</name>
    <rollno>....</rollno>
  </student>
</department>
```

Elements vs. Attributes

example-1

```
<department>
```



```

<student gender="female">
  <name>.....</name>
<rollno>....</rollno>
</student>
</department>

```

example-2

```

<department>
  <student>
    <name>.....</name>
  <rollno>....</rollno>
  <gender>female</gender>
</student>
</department>

```

In the first example, gender is an attribute. In the second example, gender is an element. Both examples provide the same information. There are no rules about when to use attributes or when to use elements in XML.

Avoid XML Attributes because they cannot contain multiple values, they cannot contain tree structures and they are not easily expandable

XML NAMESPACE

XML Namespaces provide a method to avoid element name conflicts.

In XML, element names are defined by the developer. This often results in a conflict when trying to mix XML documents from different XML applications.

<table>	<table>
<tr>	<name>African Coffee Table</name>
<td>Apples</td>	<width>80</width>
<td>Bananas</td>	<length>120</length>
</tr>	</table>
</table>	

If these XML fragments were added together, there would be a name conflict. Both contain a <table> element, but the elements have different content and meaning.

A user or an XML application will not know how to handle these differences.

Name conflicts in XML can easily be avoided using a name prefix.

```

<h:table>
<h:tr>
<h:td>Apples</h:td>
<h:td>Bananas</h:td>
</h:tr>
</h:table>
<f:table>
<f:name>African Coffee Table</f:name>
<f:width>80</f:width>
<f:length>120</f:length>
</f:table>

```

Entities

Some characters have a special meaning in XML, like the less than sign (<) that defines the start of an XML tag.

Most of you know the HTML entity: " ". This "no-breaking-space" entity is used in HTML to insert an extra space in a document. Entities are expanded when a document is parsed by an XML parser.

The following entities are predefined in XML:

Entity References	Character
<	<
>	>
&	&
"	"
'	'

PCDATA IN XML

- XML Parsed Character Data (PCDATA) is defined as a parser reads a text in the XML document, means it parses every character and determines the exact meanings.
- The element content of DTD is defined as PCDATA which parse the content of the element which is a plain text and gives the result as mark-up.
- `#PCDATA` means "Parsed Character Data", allowing an element to contain text that the XML parser will check and interpret. It can include entities, text, and valid XML syntax. It is used in

<!ELEMENT> declarations to define elements that hold readable text. PCDATA can be mixed with child elements too. It is parsed by the XML processor for well-formedness . Example in the next slide

Syntax

```
<!DOCTYPE article [
  <!ELEMENT article (title, content)>
  <!ELEMENT title (#PCDATA)>
  <!ELEMENT content (#PCDATA)>
]>
```

```
<article>
```

```
  <title>Understanding XML</title>
```

```
  <content>
```

XML stands for eXtensible Markup Language. It is used to store and transport data. The data is readable by both humans and machines.

```
</content>
```

```
</article>
```

Output:

XML stands for eXtensible Markup Language. It is used to store and transport data. The data is readable by both humans and machines.

CDATA

- XML Character data (CDATA) is defined as Blocks of texts and a type of XML Node recognized by the mark-up languages but are not parsed by the parsers.
- This is used to solve the inclusion of the mathematical term in the XML document. To pass a math equation <,>
- CDATA is used to include in the code section. .
- CDATA sections in XML contain text that should not be parsed by the XML processor. Everything inside <![CDATA[...]> is treated as plain text, even if it looks like markup. This is useful for including special characters like <, >, or & without escaping. DTD itself doesn't directly define CDATA sections, but XML elements may contain them. CDATA is ideal for code, scripts, or special symbols.
-

Syntax

```
<![CDATA[
```

```
// CDATA CODE with characters
```

```
]]>
```

Code:

```
<!DOCTYPE code [
  <!ELEMENT code (#PCDATA)>
]>
<code>
  <![CDATA[
    function greet() {
      console.log("Welcome <user> & enjoy!");
    }
  <script>
    alert("Special chars < > & stay safe!");
  </script>
  ]]>
</code>
```

DOCUMENT TYPE DEFINITION

The XML Document Type Definition, commonly known as DTD, is a way to describe XML language precisely. DTDs check vocabulary and validity of the structure of XML documents against grammatical rules of appropriate XML language.

An XML DTD can be either specified inside the document, or it can be kept in a separate document and then linked separately.

Syntax

Basic syntax of a DTD is as follows –

```
<!DOCTYPE element DTD identifier
[
  declaration1
  .....
]>
```

In the above syntax,

- The **DTD** starts with <!DOCTYPE delimiter.
- An **element** tells the parser to parse the document from the specified root element.

- **DTD identifier** is an identifier for the document type definition, which may be the path to a file on the system or URL to a file on the internet. If the DTD is pointing to external path, it is called **External Subset**.
- **The square brackets []** enclose an optional list of entity declarations called *Internal Subset*.

Internal DTD

A DTD is referred to as an internal DTD if elements are declared within the XML files. To refer it as internal DTD, *standalone* attribute in XML declaration must be set to **yes**. This means, the declaration works independent of an external source.

Syntax

Following is the syntax of internal DTD –

```
<!DOCTYPE root-element [element-declarations]>
```

where *root-element* is the name of root element and *element-declarations* is where you declare the elements.

Example

Following is a simple example of internal DTD –

```
<?xml version = "1.0" encoding = "UTF-8" standalone = "yes" ?>
```

```
<!DOCTYPE address [
```

```
  <!ELEMENT address (name,company,phone)>
```

```
  <!ELEMENT name (#PCDATA)>
```

```
  <!ELEMENT company (#PCDATA)>
```

```
  <!ELEMENT phone (#PCDATA)>
```



```
<address>
```

```
  <name>Tanmay Patil</name>
```

```
  <company>TutorialsPoint</company>
```

```
  <phone>(011) 123-4567</phone>
```

```
</address>
```

Let us go through the above code –

Start Declaration – Begin the XML declaration with the following statement.

```
<?xml version = "1.0" encoding = "UTF-8" standalone = "yes" ?>
```

DTD – Immediately after the XML header, the *document type declaration* follows, commonly referred to as the DOCTYPE –

```
<!DOCTYPE address [
```

The DOCTYPE declaration has an exclamation mark (!) at the start of the element name. The DOCTYPE informs the parser that a DTD is associated with this XML document.

DTD Body – The DOCTYPE declaration is followed by body of the DTD, where you declare elements, attributes, entities, and notations.

```
<!ELEMENT address (name,company,phone)>
```

```
<!ELEMENT name (#PCDATA)>
```

```
<!ELEMENT company (#PCDATA)>
```

```
<!ELEMENT phone_no (#PCDATA)>
```

Several elements are declared here that make up the vocabulary of the <name> document. <!ELEMENT name (#PCDATA)> defines the element *name* to be of type "#PCDATA". Here #PCDATA means parseable text data.

End Declaration – Finally, the declaration section of the DTD is closed using a closing bracket and a closing angle bracket (]>). This effectively ends the definition, and thereafter, the XML document follows immediately.

Rules

- The document type declaration must appear at the start of the document (preceded only by the XML header) – it is not permitted anywhere else within the document.
- Similar to the DOCTYPE declaration, the element declarations must start with an exclamation mark.
- The Name in the document type declaration must match the element type of the root element.

External DTD

In external DTD elements are declared outside the XML file. They are accessed by specifying the system attributes which may be either the legal *.dtd* file or a valid URL. To refer it as external

DTD, *standalone* attribute in the XML declaration must be set as **no**. This means, declaration includes information from the external source.

Syntax

Following is the syntax for external DTD –

```
<!DOCTYPE root-element SYSTEM "file-name">
```

where *file-name* is the file with *.dtd* extension.

Example

The following example shows external DTD usage –

```
<?xml version = "1.0" encoding = "UTF-8" standalone = "no" ?>
```

```
<!DOCTYPE address SYSTEM "address.dtd">
```

```
<address>
```

```
  <name>Tanmay Patil</name>
```

```
  <company>TutorialsPoint</company>
```

```
  <phone>(011) 123-4567</phone>
```

```
</address>
```

The content of the DTD file **address.dtd** is as shown –

```
<!ELEMENT address (name,company,phone)>
```

```
<!ELEMENT name (#PCDATA)>
```

```
<!ELEMENT company (#PCDATA)>
```

```
<!ELEMENT phone (#PCDATA)>
```

Merits of DTD

1. DTDs are used to define the structural components of XML document.
2. These are relatively simple and compact.
3. DTDs can be defined inline and hence can be embedded directly in the XML document.

Demerits of DTD

1. The DTDs are very basic and cannot be much specific for complex documents.

2. The language that DTD uses is not an XML document. Hence various frameworks used by XML cannot be supported by the dTDs.
3. The DTD cannot define the type of data contained within the XML document. Hence using DTD we cannot specify whether the element is numeric or string or of data type.
4. There are some XML processor which do not understand DTDs.
5. The DTDs are not aware of namespace concept.

XML SCHEMA

The short name for XML Schema is XSD(XML schema definition). An XML schema is a document or a set of documents that defines the structure and legal elements of the XML file. XML schemas can be used to ensure that an XML file is syntactically correct and conforms to the defined schema.

XML Schema have a number of advantages over DTDs.

- Schema use XML syntax, so the same tools for processing XML data (e.g., parsers, DOM, SAX, etc.) can be used for processing Schema.
- Schema support a richer set of data types, allows users to derive their own data types.
- Schema allow more flexible and powerful definitions of content models.
- Schema are extensible, supporting easier reuse of parts of schemas in other schemas (e.g., easier use of Namespaces).
- Schema have annotation elements that provide greater documentation of structural design.

<schema> Element

The <schema> element is the root element of every XML Schema. The <schema> element may contain some attributes. A schema declaration often looks something like this:

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
```

```
...
```

```
...
```

```
</xs:schema>
```

indicates that the elements and data types used in the schema come from the "http://www.w3.org/2001/XMLSchema" namespace.

Referencing a Schema in an XML file

```
<root xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
```



```
xsi:schemaLocation="file_name.xsd">
```

```
<child>
```

```
<subchild>.....</subchild>
```

```
</child>
```

```
</root>
```

Elements declarations

here are two types of Elements declarations in XML schema.

- simpleType
- complexType

simpleType

A simple element is an XML element that can contain only text. It cannot contain any other elements or attributes.

Syntax

```
<xs:element name="element_name" type="data_type" minOccurs="value" maxOccurs="value" />
```

NOTE: minOccurs and maxOccurs specify how many occurrences of the element may appear in an XML document. The default value for minOccurs is 1 and unbounded is used to specify an upper limits in maxOccurs.

example

```
<xs:element name="name" type="xs:string"/>
```

```
<xs:element name="rollno" type="xs:integer"/>
```

complexType

The complexType allows you to hold multiple elements. It can contain additional sub elements or can be left empty.

Syntax

```
<xs:element name="parent_element_name">
```

```
<xs:complexType>
```

```
<xs:sequence>
```

```
<xs:element name="child_element_name" type="data_type"/>
```

```
<xs:element name="child_element_name" type="data_type"/>
```

```
</xs:sequence>
```

```
</xs:complexType>
```

```
</xs:element>
```

example

```
<xs:element name="student" maxOccurs="unbounded">
```

```
<xs:complexType>
```

```
<xs:sequence>
```

```
<xs:element name="name" type="xs:string"/>
```

```
<xs:element name="rollno" type="xs:integer"/>
```

```
</xs:sequence>
```

```
</xs:complexType>
```

```
</xs:element>
```

Datatypes in XSD

XML Schema has a lot of built-in data types. The most common types are:

- xs:string
- xs:decimal
- xs:integer
- xs:boolean
- xs:date
- xs:time

Attributes declarations

All attributes are declared as simple types. If an element has attributes, element is considered to be of a complex type.

Syntax

```
<xs:attribute name="attribute_name" type="data_type" />
```

Default Values for Attributes

Attributes may have a default value. A default value is automatically assigned to the attribute when no other value is specified.

Syntax

```
<xs:attribute name="attribute_name" type="data_type" default="value"/>
```

Fixed Values for Attributes

Attributes may have a fixed value specified. A fixed value is also automatically assigned to the attribute, and you cannot specify another value.

Syntax

```
<xs:attribute name="attribute_name" type="data_type" fixed="value"/>
```

Optional and Required Attributes

Attributes are optional by default. To specify that the attribute is required, use the "**use**" attribute:

Syntax

```
<xs:attribute name="attribute_name" type="data_type" use="required"/>
```

XML Schema Example-1

Example: **StudentSchema.xsd**

```
<?xml version="1.0"?>

<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">

  <xs:element name="Student">

    <xs:complexType>

      <xs:sequence>

        <xs:element name="name" type="xs:string"/>

        <xs:element name="address" type="xs:string"/>

        <xs:element name="std" type="xs:string"/>

        <xs:element name="marks" type="xs:string"/>

      </xs:sequence>

    </xs:complexType>

  </xs:element>

</xs:schema>
```

</xs:complexType>

</xs:element>

</xs:schema>

SimpleXml.xml

<?xml version="1.0" encoding="UTF-8">?

<Student>

<name>Anand</name>

<address>Hyderabad</address>

<std>Tenth</std>

<marks>90 percent</marks>

</Student>

XHTML

- Stands for eXtensible HyperText Markup Language.
- A stricter version of HTML that follows XML rules.
- All tags must be lowercase and properly closed.
- Attribute values must always be quoted.
- Ensures better compatibility with XML tools and strict error checking.

The Most Important Differences from HTML:

Document Structure

XHTML DOCTYPE is mandatory

The xmlns attribute in <html> is mandatory

<html>, <head>, <title>, and <body> are mandatory

XHTML Elements

XHTML elements must be properly nested XHTML elements must always be closed XHTML elements must be in lowercase

XHTML documents must have one root element

XHTML Attributes

Attribute names must be in lower case

Attribute values must be quoted

Attribute minimization is forbidden

How to Convert from HTML to XHTML

- Add an XHTML `<!DOCTYPE>` to the first line of every page
- Add an `xmlns` attribute to the `html` element of every page
- Change all element names to lowercase
- Close all empty elements
- Change all attribute names to lowercase
- Quote all attribute values

XHTML Elements Must Be Properly Nested

```
<b><i>This text is bold and italic</i></b>
```

XHTML Elements Must Always Be Closed

```
<p>This is a paragraph</p>
```

```
<p>This is another paragraph</p>
```

Empty Elements Must Also Be Closed

A break: `<br />`

A horizontal rule: `<hr />`

An image: ``

Comparison Table of HTML/XML/XHTML

| Feature | HTML | XML | XHTML |
|--------------|-------------------|----------------------|---------------------------|
| Purpose | Display web pages | Store/transport data | Web pages using XML rules |
| Tag Names | Predefined | Custom | Predefined (HTML + XML) |
| Tag Case | Flexible | Case-sensitive | Lowercase only |
| Closing Tags | Optional | Mandatory | Mandatory |
| Structure | Loose | Strict (well-formed) | Strict (well-formed) |

| | | | |
|-----------------|-----|----|-----|
| Use in Browsers | Yes | No | Yes |
|-----------------|-----|----|-----|

PARSING XML DATA

The primary goal of any XML processor is to parse the given XML document. Java has a rich source of in-built APIs for parsing the given XML document. It is parse in two ways-

- Tree based parsing
- Event based parsing

DOM is used to parse the XML document using the tree structure.

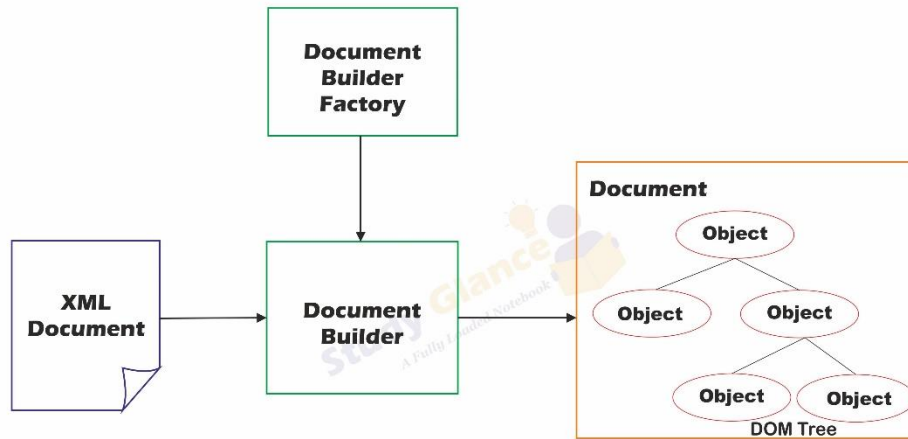
SAX allows us to access the information of XML document using sequences of events

DOM Parser

DOM stands for Document Object Model. The DOM is a programming interface for HTML(HyperText Markup Language) and XML(Extensible markup language) documents. It defines the logical structure of documents and the way a document is accessed and manipulated. The DOM API provides the classes to read and write an XML file. DOM reads an entire document. It is useful when reading small to medium size XML files. It is a tree-based parser and occupies more space when loaded into memory. We can insert and delete nodes using the DOM API.

Important features of XML DOM

1. Any node can have only one parent node. There cannot be a two-parent node for a single node.
2. A parent node can have more than one child nodes though.
3. Child nodes can have their child nodes assigned in any numbers.
4. Child nodes can have other types of nodes assigned to them called “attribute” nodes. These nodes are used to contain some extra characteristics of the node. This node is different than the child node.
5. The nodes of the same level in a tree are called “sister” nodes.
6. DOM identifies the structure of information stored in a hierarchy.
7. DOM is used to establish a relationship between data stored in different nodes.



DOM interfaces

The DOM defines several Java interfaces. Here are the most common interfaces –

- **Node** – The base datatype of the DOM.
- **Element** – The vast majority of the objects you'll deal with are Elements.
- **Attr** – Represents an attribute of an element.
- **Text** – The actual content of an Element or Attr.
- **Document** – Represents the entire XML document. A Document object is often referred to as a DOM tree.

Common DOM methods

When you are working with DOM, there are several methods you'll use often –

- **Document.getDocumentElement()** – Returns the root element of the document.
- **Node.getFirstChild()** – Returns the first child of a given Node.
- **Node.getLastChild()** – Returns the last child of a given Node.
- **Node.getNextSibling()** – These methods return the next sibling of a given Node.
- **Node.getPreviousSibling()** – These methods return the previous sibling of a given Node.
- **Node.getAttribute(attrName)** – For a given Node, it returns the attribute with the requested name.

Example:

Write a java program which takes users id as input and returns the users details by taking the user information from the XML document using DOM parser.

student.xml

<students-details>

```
<student>

<studentid>561</studentid>

<name>Ramu</name>

<address>ECIL</address>

<gender>Male</gender>

</student>

<student>

<studentid>562</studentid>

<name>Ramya</name>

<address>KBHP</address>

<gender>Female</gender>

</student>

<student>

<studentid>563</studentid>

<name>Mahi</name>

<address>BHEL</address>

<gender>Male</gender>

</student>

</students-details>
```

SAXParserxml.java

```
import org.w3c.dom.*;

import javax.xml.parsers.*;

import java.util.Scanner;

import java.io.*;

public class DOMParserDemo {public static void main(String[] args) throws Exception{
```



```
//Create a DocumentBuilder

DocumentBuilderFactory factory = DocumentBuilderFactory.newInstance();

DocumentBuilder builder = factory.newDocumentBuilder();    // parser is created

Document doc = builder.parse("student.xml"); // file is parsed


NodeList list = doc.getElementsByTagName("student");

Scanner in = new Scanner(System.in);

System.out.print("Enter student ID:\t");

int n = in.nextInt();

int flag = 0;

for(int i = 0; i < list.getLength(); i++){

    Node node = list.item(i);

    if(node.getNodeType() == Node.ELEMENT_NODE){

        Element e = (Element) node;

        int x=Integer.parseInt(e.getElementsByTagName("studentid").item(0).getTextContent());

        if(x==n){            System.out.println("\n\n STUDENT-DETAILS");

            System.out.println("=====");

            System.out.println("student id :\t" +
e.getElementsByTagName("studentid").item(0).getTextContent());

            System.out.println("student Name :\t" +
e.getElementsByTagName("name").item(0).getTextContent());

            System.out.println("Adress :\t" +
e.getElementsByTagName("address").item(0).getTextContent());

            System.out.println("Gender :\t" +
e.getElementsByTagName("gender").item(0).getTextContent());

            flag=1;

            break;
        }
    }
}
```

```

    }

    else{

        flag=0;

    }

}

}

if(flag==0)

System.out.println("student Id is not present.Try Again!!!");

}

}

```

Output

Enter student ID: 563

STUDENT-DETAILS

=====

student id : 563

student Name :Mahi

Adress : BHEL

Gender : Male

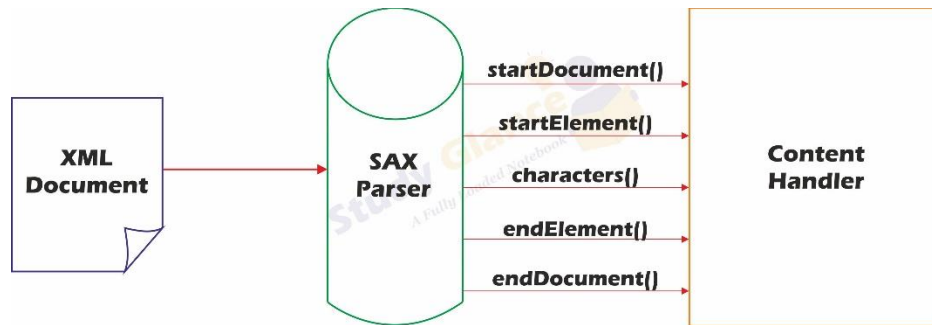
SAX Parser

SAX represents a simple API for XML and a SAX API is implemented by SAX Parser. which is an event-driven online algorithm for parsing XML documents. SAX is a way of reading data from an XML document that is an alternative to the Document Object Model(DOM). It is suitable for large XML files because it doesn't require loading the whole XML file.

We can only read and can't write the XML documents using SAX parser. It only process the XML document from top to bottom and not provide the random access facility.

SAX parser events

- startDocument
- startElement
- characters
- endElement
- endDocument



Example:

Write a java program which takes users id as input and returns the users details by taking the user information from the XML document using SAX parser.

student.xml

```

<students-details>

<student>

<studentid>561</studentid>

<name>Ramu</name>

<address>ECIL</address>

<gender>Male</gender>

</student>

<student>

<studentid>562</studentid>

<name>Ramya</name>

<address>KBHP</address>

<gender>Female</gender>
  
```

```

</student>

<student>

<studentid>563</studentid>

<name>Mahi</name>

<address>BHEL</address>

<gender>Male</gender>

</student>

</students-details>

```

SAXParserxml.java

```

import java.io.*;

import javax.xml.parsers.SAXParser;

import javax.xml.parsers.SAXParserFactory;

import org.xml.sax.Attributes;

import org.xml.sax.SAXException;

import org.xml.sax.helpers.DefaultHandler;

public class SAXParserxml extends DefaultHandler

{

    boolean studentid = true, name = false, address = false, gender = false;

    int flag=0, c=0;

    String sid, sname, sadd, sgender, tid;

    public void startDocument()

    {

        System.out.println("begin parsing document");

        System.out.print("Enter student ID:\t");

        try{

            BufferedReader reader = new BufferedReader(new InputStreamReader(System.in));

```

```

        tid = reader.readLine();
    }catch(Exception e){}
}

public void startElement(String url,String localname, String qName, Attributes att){
    if (qName.equalsIgnoreCase("studentid"))
    {
        studentid = true;
    }
    else if (qName.equalsIgnoreCase("name") && flag==1)
    {
        name = true;
    }
    else if (qName.equalsIgnoreCase("address")&& flag==1)
    {
        address = true;
    }
    else if (qName.equalsIgnoreCase("gender")&& flag==1)
    {
        gender = true;
    }
}

public void characters(char[] ch,int start,int length){
    if (studentid)
    {
        String x=new String(ch, start, length);
        if(x.equals(tid))
        {
            flag=1;sid=x; c=1;
        }
        else
            flag=0;
        studentid = false;
    }
    else if (name)
    {

```

```

        sname=new String(ch, start, length); name = false;
    }
    else if (address)
    {
        sadd=new String(ch, start, length); address = false;
    }
    else if (gender)
    {
        sgender=new String(ch, start, length); gender = false;
    }
}

```

```

public void endElement(String url,String localname, String qName){}

```

```

public void endDocument()
{
    if(c==0)
        System.out.println("student Id is not present.Try Again!!!");
    else
    {
        System.out.println("\n\n STUDENT-DETAILS");
        System.out.println("=====");
        System.out.println("student id :\t" +sid);
        System.out.println("student Name :\t" +sname);
        System.out.println("Adress :\t" +sadd);
        System.out.println("Gender :\t" +sgender);
    }
}

```

```

public static void main(String[] arg)throws Exception{
    SAXParser p=SAXParserFactory.newInstance().newSAXParser();
    p.parse(new FileInputStream("student.xml"), new SAXParserxml());
}
}

```

Output

begin parsing document

Enter student ID: 563

STUDENT-DETAILS

student id : 563

student Name :Mahi

Adress : BHEL

Gender : Male

DOM vs SAX Parser

DOM Parser

Features Of DOM Parser

- The internal structure can be created by DOM Parser.
- Because of these internal structures, the client can get information about the original XML docs.

Advantages Of DOM Parser

- DOM API is easy to use so that we can do both write and read operations.
- When a document is required then it preferred a wide part that can be randomly accessed.

Disadvantages Of DOM Parser

- Its efficiency of memory is not too good, it takes more memory cause XML docs needed to load in there.
- In comparison to the SAX parser, it is too slow.

SAX Parser

Features Of SAX Parser

- The internal structure can not be created by SAX Parser.
- These event-based SAX parsers work the same as the event handler in Java.

Advantages Of SAX Parser

- Very simple to use and has good efficiency of memory.
- Its runtime is too fast and it can work for a bigger document or file system.

Disadvantages Of SAX Parser

- Its ability to understand APIs is too less than an event-based API.
- We can't know the full information because of a lot of pieces of data.

Differences between SAX Parser and DOM Parser in Java

| DOM Parser | SAX Parser |
|---|--|
| It is called as Document Object Model. | It is called a Simple API for XML Parsing. |
| It stays in a tree structure. | It's an event-based parser. |
| DOM Parser is faster than SAX Parser. | SAX Parser is slower than DOM Parser. |
| Best for the smaller size of files. | Best for the larger sizes of files. |
| It is not good at making XML files in low memory. | It is suitable for making XML files in Java. |
| The internal structure can be created by DOM Parser. | The internal structure can not be created by SAX Parser. |
| It can insert or delete nodes. | It is read-only. |
| In DOM parser backward and forward search is possible | In the SAX parser backward navigation is not possible. |
| Suitable for large XML document. | Suitable for efficient memory. |
| It loads whole XML documents in memory. | A small part of the XML file is only loaded in memory. |